

UrbanSprout

Historias de Usuario — MVP + Production-Ready

E-commerce B2C · Kits de cultivo urbano · Stripe · Clerk · Bun + SQLite

Grupo: [Completar nombre del grupo] | Parcial #2 — 2026

Asignatura: Desarrollo de Software IX · 1GS242

Contenido

1. Inventario MVP — Funcionalidades implementadas (HU-001 a HU-024)
 2. Production-Ready — Funcionalidades faltantes (HU-025 a HU-068)
- [MCP] Historias expuestas vía Model Context Protocol
3. Anexo: Resumen de herramientas MCP

1. Inventario MVP — Funcionalidades Implementadas

HU-001 — Como visitante, quiero registrarme e iniciar sesión con Clerk (Google, Microsoft, OTP), para acceder a mi dashboard y comprar.

- CA-1: El formulario de registro y login se integra con Clerk y ofrece al menos 2 proveedores sociales.
- CA-2: Al autenticarse, el usuario es redirigido al dashboard con su sesión activa.
- CA-3: El token de Clerk se valida en cada solicitud a rutas protegidas.

HU-002 — Como usuario autenticado, quiero gestionar mi perfil desde un panel de usuario embebido de Clerk, para mantener mis datos actualizados.

- CA-4: El perfil permite editar nombre, email, foto y métodos de autenticación.
- CA-5: Los cambios persisten en Clerk y se reflejan al volver al dashboard.

HU-003 — Como administrador, quiero ser identificado por rol mediante publicMetadata de Clerk o lista blanca de emails, para acceder al panel de administración.

- CA-6: El rol se determina por `clerk.publicMetadata.role === 'admin'` o por email en `ADMIN_EMAILS`.
- CA-7: El dashboard del admin muestra un enlace al backoffice.
- CA-8: El backoffice rechaza acceso si el usuario no tiene rol admin.

HU-004 — Como visitante, quiero ver una landing page con hero, propuesta de valor, estadísticas y CTA, para entender qué ofrece UrbanSprout.

- CA-9: La landing incluye hero con CTA principal, sección de estadísticas animadas y marquee promocional.
- CA-10: Las animaciones (GSAP + Lenis) se ejecutan sin errores en Chrome y Firefox.

HU-005 — Como visitante, quiero navegar el catálogo de productos con animaciones 3D (VanillaTilt), para conocer los kits disponibles.

- CA-11: El catálogo muestra 3 productos con imagen, nombre, precio y etiqueta (Inicio / Más vendido / Premium).
- CA-12: Cada tarjeta responde con efecto tilt 3D al pasar el mouse.

HU-006 — Como visitante, quiero ver la página de detalle de cada producto (especificaciones, contenido, instrucciones), para decidir mi compra.

- CA-13: La ruta `/producto/:id` muestra breadcrumb, especificaciones, lista de incluidos, pasos, testimonial y productos relacionados.
- CA-14: El botón 'Agregar al carrito' agrega el producto al carrito contextual.

HU-007 — Como cliente, quiero agregar múltiples productos a un carrito de compras persistente, para comprar varios kits en una sola transacción.

- CA-15: El carrito se persiste en `localStorage` y sobrevive a recargas de página.
- CA-16: Se puede incrementar, decrementar y eliminar items individualmente.
- CA-17: El badge del carrito refleja la cantidad total de productos.

HU-008 — Como cliente, quiero ver un resumen del carrito con barra de envío gratis y opción de deshacer eliminaciones, para gestionar mi compra visualmente.

- CA-18: El carrito lateral (drawer) se abre con animación GSAP desde la derecha.
- CA-19: Muestra subtotal, barra de progreso para envío gratis (> \$55 USD) y botón de pago.
- CA-20: Al eliminar un producto aparece un toast 'Deshacer' por 5 segundos.

HU-009 — Como cliente, quiero pagar con Stripe Checkout (unitario o multi-item), para completar mi compra de forma segura.

- CA-21: El botón de pago crea una Stripe Checkout Session vía POST /api/checkout o /api/cart-checkout.
- CA-22: Stripe Checkout redirige al usuario a la página de éxito o cancelación.
- CA-23: El checkout rechaza si Stripe no está configurado, el producto no existe o el producto está inactivo.

HU-010 — Como cliente, quiero ver una página de éxito con confetti y pasos de progreso al completar el pago, para confirmar que la transacción fue exitosa.

- CA-24: La página /checkout/success muestra animación de confetti, resumen y pasos siguientes.
- CA-25: El parámetro ?session_id se valida y muestra datos coherentes.

HU-011 — Como cliente, quiero ver una página informativa si cancelo el pago, para entender qué ocurrió y poder reintentar.

- CA-26: La página /checkout/cancelled lista razones comunes de cancelación.
- CA-27: Incluye botón para volver al catálogo o reintentar la compra.

HU-012 — Como cliente, quiero ver el historial de mis órdenes en el dashboard con su estado actual y monto, para dar seguimiento a mis compras.

- CA-28: El dashboard lista órdenes del usuario con ID, producto, monto, estado y fecha.
- CA-29: Cada orden muestra un badge de color según su estado (pending/paid/cancelled).
- CA-30: Las órdenes pendientes se sincronizan con Stripe automáticamente.

HU-013 — Como administrador, quiero listar y gestionar órdenes desde un panel backoffice independiente, para administrar los pedidos.

- CA-31: El backoffice (app React independiente, puerto 5173) lista todas las órdenes con ID, producto, cliente, monto, estado y fecha.
- CA-32: Se puede cambiar el estado de cada orden mediante un dropdown.

HU-014 — Como administrador, quiero cambiar el estado operativo de las órdenes (pending/paid/cancelled), para reflejar el progreso de cada pedido.

- CA-33: El cambio de estado se persiste vía PATCH /orders/:id.
- CA-34: El nuevo estado se refleja inmediatamente en la UI del backoffice.

HU-015 — Como administrador, quiero gestionar el inventario (stock y stock mínimo por SKU), para mantener la disponibilidad de productos.

- CA-35: La tabla de inventario muestra SKU, stock actual, stock mínimo y alerta visual si stock < mínimo.
- CA-36: Los valores se actualizan vía PATCH /inventory/:sku.

HU-016 — Como administrador, quiero crear, editar, desactivar y eliminar productos, para mantener actualizado el catálogo.

- CA-37: El formulario de producto permite name, description, price_usd, tag e is_active.
- CA-38: Eliminar un producto con órdenes asociadas es bloqueado (se sugiere desactivar).

HU-017 — Como sistema, quiero recibir webhooks de Stripe con validación de firma para registrar órdenes automáticamente al confirmarse un pago.

- CA-39: El endpoint POST /webhooks/stripe valida la firma HTTP con stripe-webhook-secret.
- CA-40: Al recibir checkout.session.completed, crea un registro en orders y actualiza checkout_attempts.
- CA-41: Los eventos duplicados se ignoran mediante deduplicación por event.id.

HU-018 — Como desarrollador, quiero consultar un endpoint de health check, para verificar que el API está operativa.

- CA-42: GET /health retorna status 200 con JSON { status: 'ok', db: 'path', uptime: '...' }.
- CA-43: El health check funciona sin autenticación.

HU-019 — Como desarrollador, quiero ejecutar el stack completo con Docker Compose (storefront + backoffice + API + SQLite), para simplificar el entorno.

- CA-44: docker compose up levanta los 3 servicios y expone storefront en :3000, backoffice en :5173, API en :4000.
- CA-45: Los datos persisten en un volumen de SQLite.

HU-020 — Como desarrollador, quiero tener tests E2E con Playwright para rutas críticas, para validar el funcionamiento general.

- CA-46: Los tests cubren home, sign-in, sign-up, dashboard, admin y API de checkout.
- CA-47: Los tests se ejecutan con npm run test:e2e sin intervención manual.

HU-021 — Como visitante, quiero experimentar animaciones suaves (GSAP, Lenis, cursor personalizado), para tener una navegación agradable.

- CA-48: El hero tiene animación de entrada con GSAP (títulos, labels flotantes, parallax).
- CA-49: El scroll suave con Lenis está activo en toda la página.
- CA-50: El cursor personalizado (dot + ring) aparece en toda la aplicación.

HU-022 — Como visitante, quiero consultar las preguntas frecuentes en formato acordeón, para resolver dudas comunes.

- CA-51: La sección FAQ contiene 5 preguntas expandibles con animación.
- CA-52: Al hacer clic en una pregunta, se expande/colapsa su respuesta.

HU-023 — Como visitante, quiero leer testimonios de otros clientes y ver estadísticas, para generar confianza en la compra.

- CA-53: La sección de social proof muestra 3 testimonios con estrellas y nombre.
- CA-54: La sección de estadísticas muestra 4 contadores animados (clientes, semanas, satisfacción, envío).

HU-024 — Como visitante, quiero acceder a las páginas legales (términos, privacidad, devoluciones), para conocer las políticas del sitio.

- CA-55: Las rutas /terminos, /privacidad y /devoluciones muestran contenido legal completo.
- CA-56: Los enlaces están disponibles en el footer del sitio.

2. Production-Ready — Funcionalidades Faltantes

A continuación se listan las historias de usuario necesarias para considerar UrbanSprout como production-ready. Las historias marcadas con **[MCP]** se exponen a través de un servidor MCP (Model Context Protocol) bajo autenticación, importables desde Claude Code o Codex.

HU-025 [MCP] — [MCP] Como administrador, quiero que los endpoints del API verifiquen roles y permisos mediante middleware de autorización, para garantizar que solo usuarios autorizados accedan a operaciones sensibles.

- CA-57: Cada endpoint protegido rechaza con 403 si el rol no tiene el permiso requerido.
- CA-58: El middleware usa el token JWT de Clerk para extraer roles y permisos.
- CA-59: La matriz de permisos es configurable desde variables de entorno o base de datos.

Tool: `authorize_user` | **Auth:** Clerk JWT (Bearer token) | **Agente:** Validar si un usuario tiene el permiso necesario para ejecutar una acción antes de proceder.

HU-026 [MCP] — [MCP] Como administrador, quiero configurar rate limiting por endpoint y por usuario, para proteger el API contra abusos y ataques de fuerza bruta.

- CA-60: Los límites se definen por ruta (ej. 100 req/min para GET /products, 10 req/min para POST /checkout).
- CA-61: Al exceder el límite, el API responde con 429 Too Many Requests y header Retry-After.
- CA-62: Las configuraciones de rate limit son ajustables sin reiniciar el servidor.

Tool: `configure_rate_limit` | **Auth:** Admin JWT | **Agente:** Consultar y modificar los límites de tasa por ruta o por usuario.

HU-027 [MCP] — [MCP] Como administrador, quiero gestionar API keys para integraciones externas, para controlar el acceso programático al sistema.

- CA-63: Se pueden crear, listar, rotar y revocar API keys desde el panel admin.
- CA-64: Cada API key tiene un nombre descriptivo, fecha de expiración y permisos asociados.
- CA-65: Las solicitudes con API key se registran en el log de auditoría.

Tool: `manage_api_keys` | **Auth:** Admin JWT | **Agente:** Crear, revocar o listar API keys del sistema.

HU-028 — Como desarrollador, quiero que todas las entradas de usuario estén sanitizadas contra XSS e inyección, para prevenir vulnerabilidades de seguridad.

- CA-66: Todo input de texto se sanitiza antes de almacenarse en la base de datos.
- CA-67: Los parámetros de URL y body se validan con esquemas estrictos.
- CA-68: No se renderiza HTML sin escapar en el frontend.

HU-029 [MCP] — [MCP] Como administrador, quiero consultar el log de auditoría de todas las operaciones administrativas, para mantener trazabilidad de acciones sensibles.

- CA-69: Se registran con timestamp, usuario, acción, recurso y detalles cada operación CRUD en backoffice.
- CA-70: Los logs son consultables por filtros de fecha, usuario y tipo de acción.
- CA-71: Los logs de auditoría son inmutables (append-only).

Tool: `query_audit_logs` | **Auth:** Admin JWT | **Agente:** Buscar y filtrar eventos de auditoría por fecha, usuario, recurso o tipo de acción.

HU-030 [MCP] — [MCP] Como administrador, quiero procesar reembolsos automáticos desde el panel backoffice, para gestionar devoluciones sin salir de la plataforma.

- CA-72: El admin selecciona una orden pagada e inicia un reembolso parcial o total.
- CA-73: El sistema llama a la API de Stripe para ejecutar el reembolso.
- CA-74: La orden se actualiza a 'refunded' y se registra el evento en auditoría.

Tool: process_refund | **Auth:** Admin JWT + Stripe API key | **Agente:** Iniciar un reembolso para una orden especificando monto y motivo.

HU-031 [MCP] — [MCP] Como administrador, quiero sincronizar el estado de las órdenes con Stripe automáticamente, para mantener consistencia entre plataformas.

- CA-75: Un job programado consulta el estado de sesiones de Stripe abiertas y actualiza órdenes locales.
- CA-76: Las discrepancias se registran en un log para revisión manual.
- CA-77: La sincronización se puede disparar manualmente desde el backoffice.

Tool: sync_orders_stripe | **Auth:** Admin JWT | **Agente:** Sincronizar el estado de órdenes locales con Stripe y reportar discrepancias.

HU-032 [MCP] — [MCP] Como cliente, quiero guardar métodos de pago en mi cuenta (SetupIntents de Stripe), para agilizar compras futuras.

- CA-78: El dashboard del cliente muestra opción 'Guardar tarjeta' que crea un SetupIntent.
- CA-79: Los métodos guardados se muestran como opción en el checkout.
- CA-80: Stripe PaymentMethod se asocia al customer_id de Clerk.

Tool: manage_payment_methods | **Auth:** User JWT | **Agente:** Listar, agregar o eliminar métodos de pago guardados del cliente.

HU-033 — Como cliente, quiero descargar facturas de mis compras en formato PDF, para tener comprobantes fiscales.

- CA-81: Cada orden pagada tiene un botón 'Descargar factura' en el dashboard.
- CA-82: La factura incluye datos del cliente, productos, montos, impuestos y fecha.
- CA-83: El PDF se genera del lado del servidor con un template estandarizado.

HU-034 [MCP] — [MCP] Como administrador, quiero centralizar los logs de la aplicación con niveles estructurados, para diagnosticar problemas en producción.

- CA-84: Los logs del API, storefront y backoffice se envían a un destino centralizado (archivo o servicio externo).
- CA-85: Cada entrada tiene nivel (info/warn/error), timestamp, servicio y mensaje estructurado.
- CA-86: Los logs son consultables por servicio, nivel y rango de fechas.

Tool: query_logs | **Auth:** Admin JWT | **Agente:** Consultar logs del sistema filtrando por servicio, nivel de severidad y rango de tiempo.

HU-035 [MCP] — [MCP] Como administrador, quiero ver un dashboard de métricas de rendimiento (tiempo de respuesta, tasa de error, uptime), para monitorear la salud del sistema.

- CA-87: El dashboard muestra gráficos de latencia promedio (P50, P95, P99) por endpoint.
- CA-88: Muestra tasa de error 4xx/5xx en tiempo real.
- CA-89: El uptime del API se muestra con indicador verde/rojo.

Tool: get_performance_metrics | **Auth:** Admin JWT | **Agente:** Consultar métricas de rendimiento agregadas del API.

HU-036 [MCP] — [MCP] Como administrador, quiero ver un dashboard de métricas de negocio (revenue, conversión, ticket promedio), para tomar decisiones comerciales.

- CA-90: El dashboard muestra revenue total y por período, tasa de conversión checkout→pago, y AOV.
- CA-91: Los datos son exportables a CSV.
- CA-92: Las métricas se actualizan en cuasi-real time con datos de webhooks.

Tool: get_business_metrics | **Auth:** Admin JWT | **Agente:** Consultar KPIs de negocio: revenue, conversión, AOV, órdenes por período.

HU-037 [MCP] — [MCP] Como desarrollador, quiero tener error tracking con contexto (usuario, acción, stack trace), para corregir errores en producción rápidamente.

- CA-93: Los errores no manejados se capturan y envían a un servicio de error tracking (Sentry o similar).
- CA-94: Cada error incluye contexto: usuario autenticado, ruta, payload, stack trace.
- CA-95: Los errores se agrupan por fingerprint y se puede marcar como resueltos.

Tool: `get_error_feed` | **Auth:** Developer API key | **Agente:** Listar errores recientes con agrupación, stack trace y contexto.

HU-038 — Como usuario, quiero ver un Error Boundary amigable cuando un componente React falle, para no perder la navegación.

- CA-96: Cada sección principal (home, productos, dashboard) tiene su propio Error Boundary.
- CA-97: El boundary muestra un mensaje claro y un botón 'Reintentar'.
- CA-98: El error se registra en consola y en el servicio de error tracking.

HU-039 — Como usuario, quiero páginas de error personalizadas (404, 500, red) con acciones sugeridas, para saber qué hacer cuando algo sale mal.

- CA-99: La página 404 muestra 'Página no encontrada' con enlace al inicio.
- CA-100: La página 500 muestra 'Error interno' con instrucciones y opción de reintentar.
- CA-101: Los errores de red muestran un mensaje de 'Sin conexión' y reintento automático.

HU-040 — Como desarrollador, quiero un formato de respuesta de error estandarizado en todo el API, para facilitar la integración con clientes.

- CA-102: Toda respuesta de error sigue el schema: { error: { code, message, details } }.
- CA-103: Los códigos de error son consistentes (INVALID_INPUT, NOT_FOUND, UNAUTHORIZED, FORBIDDEN, RATE_LIMITED).
- CA-104: El campo details incluye información útil para depuración cuando es seguro.

HU-041 — Como cliente, quiero validación en tiempo real en todos los formularios (registro, checkout, perfil), para corregir datos antes de enviar.

- CA-105: Los campos obligatorios muestran error inline al perder el foco si están vacíos.
- CA-106: El email y datos de pago se validan con formato antes del submit.
- CA-107: El botón de submit se deshabilita si hay errores de validación.

HU-042 [MCP] — [MCP] Como desarrollador, quiero tests unitarios automatizados para la lógica de negocio del API, para garantizar corrección al refactorizar.

- CA-108: Las funciones de cálculo de precios, validación de productos y reglas de inventario tienen cobertura $\geq 80\%$.
- CA-109: Los tests se ejecutan en el pipeline de CI.
- CA-110: Cada test es independiente y no requiere base de datos real (mocks/stubs).

Tool: `run_unit_tests` | **Auth:** CI API key | **Agente:** Ejecutar la suite de tests unitarios y devolver resultados (passed/failed/coverage).

HU-043 [MCP] — [MCP] Como desarrollador, quiero tests de integración para todos los endpoints del API, para validar el comportamiento completo del sistema.

- CA-111: Cada endpoint tiene al menos un test de caso feliz y uno de caso error.
- CA-112: Los tests de integración usan una base de datos SQLite de prueba separada.
- CA-113: Se prueban especialmente los flujos de checkout, webhooks y roles.

Tool: `run_integration_tests` | **Auth:** CI API key | **Agente:** Ejecutar tests de integración contra entorno de staging/CI y reportar resultados.

HU-044 — Como desarrollador, quiero tests de componentes para componentes React críticos, para prevenir regresiones visuales y funcionales.

- CA-114: Componentes como CartDrawer, ProductCard y CheckoutButton tienen tests con React Testing Library.

- CA-115: Los tests cubren renderizado, interacciones del usuario y estados vacío/error.
- CA-116: Se integran en el pipeline de CI.

HU-045 — Como desarrollador, quiero tests de accesibilidad automatizados en el pipeline, para garantizar que el sitio sea inclusivo.

- CA-117: Se ejecuta aXe o Lighthouse CI en las rutas principales.
- CA-118: No se permiten regresiones en los scores de accesibilidad.
- CA-119: Los issues se reportan como warnings en el PR.

HU-046 [MCP] — [MCP] Como desarrollador, quiero un pipeline de CI (GitHub Actions) que ejecute lint, typecheck y tests, para detectar errores antes del merge.

- CA-120: El pipeline se dispara en cada push y pull request a main.
- CA-121: Ejecuta eslint, tsc --noEmit, tests unitarios y tests de integración.
- CA-122: El pipeline falla si algún paso da error y bloquea el merge.

Tool: trigger_ci_pipeline | **Auth:** GitHub webhook secret / CI token | **Agente:** Disparar el pipeline CI y monitorear su resultado hasta completar.

HU-047 [MCP] — [MCP] Como desarrollador, quiero un pipeline de CD automatizado que despliegue a staging/producción tras pasar los tests, para liberar cambios continuamente.

- CA-123: Tras merge a main con CI exitoso, se despliega automáticamente a staging.
- CA-124: El despliegue a producción requiere aprobación manual.
- CA-125: Cada despliegue genera un tag de versión y changelog automático.

Tool: trigger_deployment | **Auth:** CI token + aprobación manual | **Agente:** Disparar despliegue a un entorno específico (staging/production) y monitorear el progreso.

HU-048 — Como administrador, quiero migraciones de base de datos versionadas, para evolucionar el esquema sin pérdida de datos.

- CA-126: Las migraciones son archivos SQL secuenciales con timestamp.
- CA-127: Se ejecutan automáticamente al iniciar el API si hay migraciones pendientes.
- CA-128: Los rollbacks están documentados y disponibles.

HU-049 — Como desarrollador, quiero configuración por entorno (dev/staging/prod) con validación de variables, para evitar errores de configuración en despliegue.

- CA-129: Las variables requeridas se validan al iniciar la aplicación.
- CA-130: Hay archivos .env.example documentados para cada entorno.
- CA-131: Los secrets se inyectan desde el orquestador/CI, no desde el código.

HU-050 [MCP] — [MCP] Como administrador, quiero un sistema de permisos granular (RBAC) con matriz de roles y acciones, para controlar qué puede hacer cada tipo de usuario.

- CA-132: Los roles disponibles incluyen: superadmin, admin, support, client.
- CA-133: Cada rol tiene permisos específicos (ej. support puede ver órdenes pero no editar productos).
- CA-134: La matriz de permisos es consultable desde la UI del backoffice.

Tool: get_role_permissions | **Auth:** Admin JWT | **Agente:** Consultar la matriz de permisos de un rol o usuario específico.

HU-051 [MCP] — [MCP] Como administrador, quiero invitar, suspender y cambiar roles de usuarios desde el panel, para gestionar el equipo administrativo.

- CA-135: El admin puede buscar usuarios por nombre/email y ver su rol actual.
- CA-136: Puede cambiar el rol de un usuario (ej. support → admin) con confirmación.
- CA-137: Puede suspender usuarios, lo que bloquea su acceso inmediatamente.

Tool: manage_users | **Auth:** Admin JWT | **Agente:** Buscar, invitar, cambiar rol o suspender usuarios del sistema.

HU-052 — Como cliente, quiero que mis datos estén aislados de otros clientes, para garantizar la privacidad de mi información.

- CA-138: Cada query de órdenes filtra por buyer_id del token autenticado.
- CA-139: Los datos de perfil no son accesibles por otros usuarios ni en listados públicos.
- CA-140: Se verifica en tests de integración que un usuario no vea datos de otro.

HU-053 [MCP] — [MCP] Como cliente, quiero buscar y filtrar productos por nombre, precio y categoría, para encontrar rápidamente lo que necesito.

- CA-141: La búsqueda soporta texto parcial y devuelve resultados en < 500ms.
- CA-142: Los filtros combinables incluyen rango de precio, categoría y etiqueta.
- CA-143: Los resultados muestran paginación de máximo 12 productos por página.

Tool: search_products | **Auth:** Público (sin autenticación requerida) | **Agente:** Buscar productos con filtros de texto, precio, categoría y paginación.

HU-054 — Como cliente, quiero navegar productos organizados por categorías y etiquetas, para descubrir productos relacionados fácilmente.

- CA-144: Cada producto pertenece a una categoría (ej. 'Kits Básicos', 'Kits Premium').
- CA-145: Las etiquetas (Inicio, Más vendido, Premium) son configurables desde el backoffice.
- CA-146: La home muestra secciones por categoría cuando hay múltiples productos.

HU-055 — Como cliente, quiero recibir notificaciones por email cuando el estado de mi orden cambie, para estar informado sin revisar el dashboard.

- CA-147: Se envía email al confirmarse el pago con resumen de la orden.
- CA-148: Se envía email cuando la orden cambia a 'enviado' o 'completado'.
- CA-149: Los emails usan templates responsivos con la marca UrbanSprout.

HU-056 — Como administrador, quiero recibir alertas automáticas cuando el inventario llegue a stock mínimo, para reabastecer a tiempo.

- CA-150: Al actualizar una orden pagada, si stock < minimum_stock se dispara alerta.
- CA-151: La alerta se muestra en el backoffice con badge rojo en la navbar.
- CA-152: Opcionalmente se envía email al admin configurado.

HU-057 — Como cliente, quiero aplicar cupones de descuento en mi carrito, para obtener mejores precios.

- CA-153: El carrito tiene un campo 'Código de descuento' que valida contra la base de datos.
- CA-154: Los cupones pueden ser porcentaje o monto fijo, con fecha de expiración.
- CA-155: El descuento se refleja en el subtotal antes de redirigir a Stripe.

HU-058 [MCP] — [MCP] Como cliente, quiero agregar productos a una lista de deseos, para guardar productos que me interesan para después.

- CA-156: Cada producto tiene un ícono de corazón para agregar/quitar de la wishlist.
- CA-157: La wishlist se muestra en el dashboard con los productos guardados.
- CA-158: Desde la wishlist se puede agregar todo al carrito con un clic.

Tool: manage_wishlist | **Auth:** User JWT | **Agente:** Agregar, quitar o consultar productos en la lista de deseos del usuario.

HU-059 — Como cliente, quiero calificar y reseñar productos que compré, para compartir mi experiencia con otros compradores.

- CA-159: Solo clientes que compraron el producto pueden reseñarlo.
- CA-160: La reseña incluye calificación (1-5 estrellas) y texto opcional.
- CA-161: Las reseñas se muestran en la página de detalle del producto.

HU-060 [MCP] — [MCP] Como visitante, quiero cambiar el idioma del sitio (español/inglés), para usar la plataforma en mi idioma preferido.

- CA-162: El selector de idioma está disponible en el header y persiste la preferencia.
- CA-163: Todas las cadenas visibles están externalizadas en archivos de traducción.
- CA-164: Al menos español e inglés están soportados con cobertura completa.

Tool: `get_translations` | **Auth:** Público | **Agente:** Obtener las traducciones para un locale específico y una sección determinada.

HU-061 [MCP] — [MCP] Como visitante con discapacidad visual, quiero que el sitio cumpla con WCAG 2.1 AA, para poder usar la plataforma de forma autónoma.

- CA-165: Contraste de color mínimo 4.5:1 en textos normales.
- CA-166: Todos los elementos interactivos son accesibles por teclado (Tab, Enter, Escape).
- CA-167: Los componentes tienen atributos ARIA apropiados (aria-label, role, live regions).
- CA-168: Las imágenes tienen texto alternativo descriptivo.

Tool: `run_accessibility_audit` | **Auth:** CI API key | **Agente:** Ejecutar una auditoría de accesibilidad (axe/Lighthouse) y reportar violaciones de WCAG.

HU-062 — Como usuario, quiero elegir entre modo claro y oscuro, para personalizar mi experiencia visual.

- CA-169: El toggle está disponible en el header y persiste la preferencia en localStorage.
- CA-170: Respetar la preferencia del sistema (prefers-color-scheme) por defecto.
- CA-171: Todos los componentes tienen variantes para ambos modos.

HU-063 [MCP] — [MCP] Como administrador, quiero programar backups automáticos de la base de datos con restauración por punto en el tiempo, para prevenir pérdida de datos.

- CA-172: Se ejecuta un backup diario automático de la base SQLite.
- CA-173: Los backups se almacenan con timestamp en un directorio configurable.
- CA-174: El admin puede listar backups disponibles y restaurar desde un punto específico.
- CA-175: La restauración requiere confirmación y muestra advertencia de pérdida de datos posteriores.

Tool: `manage_backups` | **Auth:** Admin JWT | **Agente:** Crear, listar y restaurar backups de la base de datos.

HU-064 [MCP] — [MCP] Como administrador, quiero exportar órdenes y productos a CSV/JSON, para integrar con sistemas externos o hacer análisis offline.

- CA-176: El admin selecciona el tipo de datos (órdenes, productos, clientes) y formato (CSV o JSON).
- CA-177: Se pueden aplicar filtros por rango de fechas y estado.
- CA-178: El archivo se descarga automáticamente con nombre descriptivo.

Tool: `export_data` | **Auth:** Admin JWT | **Agente:** Exportar datos del sistema en formato CSV o JSON con filtros opcionales.

HU-065 — Como usuario, quiero tener herramientas de cumplimiento GDPR (exportar mis datos, eliminar mi cuenta), para ejercer mis derechos de privacidad.

- CA-179: El dashboard tiene opción 'Exportar mis datos' que genera un JSON con toda la información del usuario.
- CA-180: La opción 'Eliminar mi cuenta' requiere confirmación y password.
- CA-181: La eliminación es irreversible y anonimiza las órdenes del usuario.

HU-066 [MCP] — [MCP] Como cliente, quiero cancelar una orden pendiente desde mi dashboard, para anular compras que aún no se procesaron.

- CA-182: Las órdenes con estado 'pending' muestran botón 'Cancelar orden'.
- CA-183: La cancelación actualiza el estado a 'cancelled' y notifica al administrador.
- CA-184: No se puede cancelar una orden ya pagada o previamente cancelada.

Tool: `cancel_order` | **Auth:** User JWT | **Agente:** Cancelar una orden del usuario si está en estado pendiente.

HU-067 [MCP] — [MCP] Como administrador, quiero generar reportes de ventas por período con desglose por producto, para analizar tendencias de negocio.

- CA-185: El reporte incluye revenue total, cantidad de órdenes, producto más vendido y ticket promedio.
- CA-186: Se puede filtrar por rango de fechas personalizado.
- CA-187: El reporte se visualiza en pantalla y es exportable a PDF o CSV.

Tool: generate_sales_report | **Auth:** Admin JWT | **Agente:** Generar un reporte de ventas para un período específico con desglose por producto.

HU-068 [MCP] — [MCP] Como administrador, quiero gestionar el catálogo completo (productos + inventario) desde herramientas MCP, para automatizar operaciones de mantenimiento.

- CA-188: Se puede consultar el listado completo de productos con stock disponible.
- CA-189: Se puede actualizar el precio o estado de un producto remotamente.
- CA-190: Se pueden crear nuevos productos con todos sus atributos.

Tool: manage_catalog | **Auth:** Admin JWT | **Agente:** Consultar, crear y actualizar productos e inventario del catálogo.

3. Anexo: Resumen de Herramientas MCP

Las siguientes herramientas MCP se exponen bajo autenticación y son importables desde Claude Code o Codex para que agentes de IA ejecuten operaciones sobre UrbanSprout.

Tool MCP	Auth	HU	Qué permite al agente
authorize_user	Clerk JWT	HU-025	Validar permisos de usuario
configure_rate_limit	Admin JWT	HU-026	Ajustar límites de tasa
manage_api_keys	Admin JWT	HU-027	Crear/revocar API keys
query_audit_logs	Admin JWT	HU-029	Buscar eventos de auditoría
process_refund	Admin JWT + Stripe	HU-030	Procesar reembolsos
sync_orders_stripe	Admin JWT	HU-031	Sincronizar órdenes con Stripe
manage_payment_methods	User JWT	HU-032	Gestionar métodos de pago
query_logs	Admin JWT	HU-034	Consultar logs del sistema
get_performance_metrics	Admin JWT	HU-035	Métricas de rendimiento
get_business_metrics	Admin JWT	HU-036	KPIs de negocio
get_error_feed	Developer API key	HU-037	Error tracking
run_unit_tests	CI API key	HU-042	Ejecutar tests unitarios
run_integration_tests	CI API key	HU-043	Ejecutar tests integración
trigger_ci_pipeline	CI token	HU-046	Disparar CI pipeline
trigger_deployment	CI token	HU-047	Disparar deployment
get_role_permissions	Admin JWT	HU-050	Consultar matriz de permisos
manage_users	Admin JWT	HU-051	Gestionar usuarios del sistema
search_products	Público	HU-053	Buscar productos con filtros
manage_wishlist	User JWT	HU-058	Gestionar lista de deseos
get_translations	Público	HU-060	Obtener traducciones i18n
run_accessibility_audit	CI API key	HU-061	Auditar accesibilidad
manage_backups	Admin JWT	HU-063	Gestionar backups DB
export_data	Admin JWT	HU-064	Exportar datos del sistema
cancel_order	User JWT	HU-066	Cancelar orden pendiente
generate_sales_report	Admin JWT	HU-067	Reporte de ventas
manage_catalog	Admin JWT	HU-068	Gestionar catálogo completo

Resumen: 24 HU implementadas (MVP) + 44 HU production-ready = 68 total. De las cuales **26** son historias [\[MCP\]](#).